Privacy-Preserving Recommendation Systems: A Survey of Cryptographic Approaches

Literature Survey, CS670

Mainak Sarkar (230619) Shrasti Dwivedi (230982) Aditya Anand (230065)
Shravan Agrawal (230984)

CS670: Cryptographic Techniques for Privacy Preservation
Indian Institute of Technology Kanpur
31 August 2026

Presentation video (unlisted, YouTube): https://youtu.be/5_uQ8ESewRk

Abstract

Recommender systems are trained on records of what people consume, and that data has been shown to be identifying as well as sensitive. A large body of work asks whether such systems can be built without requiring their operator to see the underlying data. The answers come from several communities: secure multi-party computation, private information retrieval, oblivious memory, federated learning, and differential privacy. These communities assume different threat models and, crucially, protect different parts of the system.

This survey organises that landscape around the recommendation *pipeline*: collection, training, serving and delivery. We propose a taxonomy along four axes: trust model, privacy notion, pipeline stage protected, and adversary model. Every surveyed system is placed on this taxonomy. We then treat the cryptographic line in detail: the building blocks, the private-training literature from garbled circuits through to three-party power iteration, and the private-retrieval literature that has developed largely in parallel under the name private nearest-neighbour search.

Three observations emerge. Approaches are consistently cheaper in proportion to the trust assumptions they make, and the fastest systems in every family assume non-colluding parties. Almost every system evaluated at realistic scale is semi-honest. And the delivery stage, which fetches the recommended item without revealing which it was, is addressed almost entirely by the private information retrieval literature in isolation from any recommender. The one system spanning both halves predates the modern private-retrieval line. We also argue that published costs in this field are not comparable across papers, and report each system's claims in the terms its authors stated them rather than constructing a ranking.

Contents

1 Introduction

Recommender systems are built from the most revealing data their users produce: what they watched, read, bought and clicked. That this data is sensitive is uncontroversial. That it is also *identifying* was established by Narayanan and Shmatikov [NS08], who de-anonymised records in the Netflix Prize dataset using the Internet Movie Database as background knowledge, and thereby uncovered "apparent political preferences and other potentially sensitive information".

A substantial body of work has since asked whether a recommender can be built that does not require its operator to see this data. The answers come from several communities: secure multi-party computation, private information retrieval, oblivious memory, federated learning, and differential privacy. These communities use different vocabulary, assume different threat models, and frequently protect different parts of the system. This survey maps that landscape.

1.1 Scope

We survey approaches to privacy-preserving recommendation with an emphasis on *cryptographic* techniques: secure multi-party computation, homomorphic encryption, private information retrieval, function secret sharing, and oblivious memory.

The taxonomy of Section 3 covers the field as a whole, including federated, differentially private and trusted-hardware approaches, so that the classification is complete and so that a reader can see how the families relate. Mechanism-level treatment in Sections 4–7 is restricted to the cryptographic line. Federated and differentially private recommenders are discussed where they bear on the cryptographic story, notably in Sections 5.4 and 9.4. They are not surveyed exhaustively, since existing surveys already cover them well. Trusted execution environments are mentioned only for completeness, as their security rests on hardware assumptions and side-channel resistance rather than on the techniques studied here.

We also do not attempt to rank systems against one another. Section 8 explains why that would be unsound.

1.2 Relation to existing surveys

The federated and differentially private branches of this field are already well surveyed, as is privacy-preserving machine learning generally. What those surveys largely do not cover is the *cryptographic* line as it applies specifically to recommendation, and in particular the *delivery* stage: retrieving the recommended item without revealing which it was. That stage is where the recommendation literature and the private-information-retrieval literature meet, and organising the survey around the pipeline (Section 2.2) is what makes the meeting point visible.

1.3 Method

Papers were selected by following three threads. The first is the two systems that address recommendation and retrieval jointly, together with their reference lists. The second is the primitives those systems are built from. The third is the private-nearest-neighbour literature that has developed alongside them. Bibliographic metadata for every cited work was taken from the DBLP computer science bibliography rather than transcribed by hand.

We distinguish explicitly between works we read in full, works for which we read the abstract, and works cited on metadata alone. Claims about the first two are made directly. For the third, we state what a source we did read reports about them, attributed. Where we could not

obtain a paper, we say so at the point of use rather than paraphrasing it. Two such cases appear below [CKN+11, NIW+13].

1.4 Organisation

Section 2 introduces collaborative filtering and the four-stage pipeline around which the survey is organised. Section 3 presents the taxonomy: trust model, privacy notion, pipeline stage and adversary, and places the surveyed systems on those axes. Section 4 covers the cryptographic building blocks. Sections 5 and 6 survey private training and private retrieval respectively, each ending with where its approaches stop. Section 7 examines the systems that span more than one stage and the obstacles to composing the rest. Section 8 describes how the field evaluates itself and why cross-paper comparison is unsound, and Section 9 collects the open problems.

2 Background

A recommender system is a data-mining pipeline whose input is, by construction, a record of what people consume. This section fixes the vocabulary the rest of the survey uses, and introduces the device around which it is organised: the recommendation *pipeline*, and the observation that different bodies of work protect different stages of it.

2.1 Collaborative filtering and matrix factorisation

Collaborative filtering predicts a user's preferences from the preferences of other users. Approaches divide broadly into neighbourhood methods, which score an item by its similarity to items the user already liked, and latent-factor models, which embed users and items in a shared low-dimensional space. Matrix factorisation is the dominant latent-factor technique [KBV09, HKV08]. Given a partially observed user–item matrix $\mathbf{U} \in \mathbb{R}^{m \times n}$, it seeks

$$\mathbf{U} \;\approx\; \mathbf{A} \cdot \mathbf{B}, \qquad \mathbf{A} \in \mathbb{R}^{m \times d}, \quad \mathbf{B} \in \mathbb{R}^{d \times n}, \quad d \ll \min(m, n),$$

so that each user and each item is described by a d-dimensional embedding, and the predicted affinity of user i for item j is the inner product of their embeddings.

Neural approaches to collaborative filtering exist and report improvements: Liang et al. [LKHJ18] extend variational autoencoders to implicit feedback, describing a "non-linear probabilistic model" that goes "beyond the limited modeling capacity of linear factor models which still largely dominate collaborative filtering research", and report outperforming several baselines including two neural ones. That claim should be read alongside Ferrari Dacrema et al. [DCJ19], whose systematic analysis of neural recommendation proposals was motivated by "problems in today's research practice . . . in terms of the reproducibility of the results or the choice of the baselines".

This tension matters for the present survey in a specific way. The cryptographic literature surveyed in Section 5 targets matrix factorisation almost exclusively, and does not attempt to run deep recommenders under secure computation. Given that matrix factorisation remains a competitive baseline, and that every additional non-linearity is expensive to evaluate securely (Section 4.7), that focus is a reasonable engineering choice rather than a gap.

2.2 The recommendation pipeline

It is useful to separate a deployed recommender into four stages:

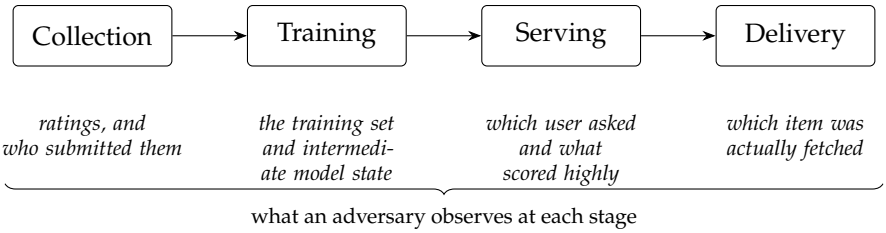

Figure 1: The recommendation pipeline. Approaches surveyed here protect different subsets of these four stages, and a pipeline is only as private as its weakest stage.

1. **Collection.** Ratings, clicks or views leave the user and reach the service.
2. **Training.** A model, here the factorisation $\mathbf{U} \approx \mathbf{A} \cdot \mathbf{B}$, is fitted to the collected data.
3. **Serving.** For a given user, scores are computed over the catalogue and a ranking is produced.
4. **Delivery.** The user fetches the recommended item itself: the film, the article, the product page.

These stages are separable, and different lines of work protect different subsets of them. That observation organises this survey, and Section 3 makes it one of the four axes of the taxonomy. It also carries an immediate consequence: *a pipeline is only as private as its weakest stage*. Training a model under secure computation achieves little if the subsequent fetch of the recommended item is performed in the clear, because the fetch reveals the recommendation that the training protocol was protecting.

2.3 What the adversary observes

Figure 1 annotates each stage with what is exposed to a service that runs it without protection. Two results establish that this exposure is a demonstrated risk rather than a hypothetical one.

Narayanan and Shmatikov [NS08] present "a new class of statistical de-anonymization attacks against high-dimensional micro-data, such as individual preferences, recommendations, transaction records". They applied it to the Netflix Prize dataset, described as "anonymous movie ratings of 500,000 subscribers", using the Internet Movie Database as background knowledge, and report having "successfully identified the Netflix records of known users, uncovering their apparent political preferences and other potentially sensitive information". They further note that the attack is "robust to perturbation in the data" and tolerates "some mistakes in the adversary's background knowledge". Ratings data, in other words, is identifying and not merely sensitive, and releasing it in a nominally anonymised form is not sufficient.

Calandrino et al. [CKN+11] go further and consider the privacy risks of collaborative filtering itself.[1] The distinction is important for Section 9.4: an attack on the *released dataset* is addressed by protecting the pipeline, whereas an attack on the recommender's *intended output* is not.

2.4 Utility, and why it constrains the cryptography

Recommendation quality is measured by ranking metrics computed over a held-out set, most commonly normalised discounted cumulative gain (nDCG@k) and recall at k. This matters here

[1] We were unable to obtain this paper: it is paywalled and no open version was found. We therefore cite it only for the claim its title makes, and do not characterise its method or results.

because a private recommender that recommends badly has solved nothing, and because the constraint propagates into protocol design: fixed-point arithmetic, approximation of non-linear functions, and reduced-precision representations must not perturb the ranking. Reporting quality alongside cost is accordingly the norm in this literature, and Section 8 returns to it.

3 A Taxonomy of Approaches

Work in this area is routinely compared as though the systems were alternatives to one another. They frequently are not: they protect different stages of the pipeline, under different trust assumptions, offering guarantees that are not of the same kind. A system requiring two non-colluding servers is not simply "weaker" than one requiring none, and a differentially private recommender is not a slower or faster version of a cryptographic one.

This section proposes four axes along which any system in the field can be placed, and then places the surveyed systems on them. The axes are what make the comparisons in Sections 5 and 6 honest.

3.1 Axis 1: the trust model

Who must be trusted, and not to do what?

Single server. No non-collusion assumption. Achieved with homomorphic encryption or single-server PIR, and paid for in server computation. SimplePIR [HHC+23], Spiral [MW22] and Panther [LHZ+25] sit here, as does Tiptoe [HDCZ23], whose authors state that its "privacy guarantee is based on cryptography alone; it does not require hardware enclaves or non-colluding servers".

Two or more non-colluding servers. The database or the secret state is replicated or shared across parties assumed not to collude. This buys dramatically cheaper protocols: multi-server PIR [CGKS95, HH19], two-server machine learning [MZ17], three-party computation [WGC19, AFL+16], and the systems of Section 5.3. The assumption is about the world, not about mathematics, and it is the single most consequential choice in this taxonomy.

Federated / user-side. No server holds raw data, and users compute locally and upload updates [AIK+19]. The guarantee is weaker than it first appears, as Section 5.4 explains.

Trusted hardware. Computation runs inside an enclave. Out of scope here (Section 1.1), and listed only for completeness.

These are not orderable by strength. They are different assumptions, and the right question is which is plausible for a given deployment.

3.2 Axis 2: the privacy notion

What does "private" formally mean?

Cryptographic. Security is defined by the real/ideal simulation paradigm [Lin17, EKR18]: a protocol is secure if whatever an adversary learns from its execution could be simulated from the agreed output alone. Nothing beyond the output leaks, but the output itself is not protected.

Differential privacy. An individual's influence on a released quantity is bounded [DR14], first applied to recommendation by McSherry and Mironov [MM09]. This constrains what the *output* reveals, and says nothing about who observes the computation. Wally [ABG+24] occupies an interesting middle position, using differential privacy "to break the linear computation barrier of fully-oblivious schemes".

Anonymity. Identity rather than content is hidden, typically by mixing or relaying. Weaker against traffic analysis, but cheap and deployed.

The critical point, and the one Section 9.4 returns to, is that these compose but do not substitute. Cryptographic privacy bounds *who sees the computation*, while differential privacy bounds *what the result implies*. A system can be cryptographically perfect and still leak, if its intended output is itself revealing.

3.3 Axis 3: which pipeline stage is protected

Mapping onto Figure 1: collection, training, serving, delivery. This is the axis the literature is least explicit about, and the one that most often makes two systems incomparable. It is also where Table 1 is most informative, because the columns are conspicuously uneven.

3.4 Axis 4: the adversary

Semi-honest adversaries follow the protocol and try to learn from what they see, while malicious adversaries may deviate arbitrarily. Orthogonally, a scheme specifies how many parties may be corrupted, and whether corruption is fixed in advance. Almost every system evaluated at realistic scale in this survey is semi-honest: Duoram [VHG23] tolerates "a single passive corruption", Prac [SVG24] assumes "none of the three parties ... collude", and Nudge [HDCB26] protects against "an adversary that compromises the entire secret state of one server". That is a substantive limitation rather than a technicality, discussed in Section 9.1.

3.5 Placing the systems

Table 1 places the surveyed *systems* on these axes. General-purpose primitives (secret sharing, garbled circuits, homomorphic encryption) are not systems in this sense and are treated in Section 4 instead.

3.6 Reading the table

Three observations follow from Table 1, and each is developed later.

The stage column is uneven. Training and serving are well populated, while *delivery* is addressed almost exclusively by the PIR line. Only one recommendation system in the table, Pirsona [VBH21], spans collection, training and delivery together. The oblivious-memory row group has no stage at all, because those works are primitives that a system must be built *from*. Section 7 takes up what this unevenness means.

Cost is bought with assumptions, consistently. The cheapest protocols in each row group are those assuming non-collusion. This is most visible in PIR, where the authors of SimplePIR [HHC+23] position their single-server throughput as approaching "the performance of the fastest two-server private-information-retrieval schemes (which require non-colluding servers)". That comparison is worth making precisely because removing the assumption normally costs something.

Table 1: Surveyed systems placed on the four axes of Section 3. Pipeline stages: Collection, Training, Serving, Delivery. Entries marked † are placed from title, venue and the description given by a source we read, as recorded in `notes/evidence.md`.

System	Trust model	Notion	Stage	Adversary
Recommendation systems				
Nikolaenko et al. [NIW+13]†	2 non-colluding	crypto	T	semi-honest
Pirsona [VBH21]	s+1 pairwise n.c.	crypto	C,T,D	semi-honest
Nudge [HDCB26]	3, ≤ 1 corrupt	crypto	C,T,S	semi-honest
Shmueli–Tassa [ST17]†	multi-party	crypto	T	semi-honest
FedMF [CWCY21]	federated + HE	crypto	C,T	semi-honest
FedCF [AIK+19]	federated	none*	C,T	—
McSherry–Mironov [MM09]†	single server	DP	T	—
Secure machine learning (recommendation-adjacent)				
SecureML [MZ17]	2 non-colluding	crypto	T	semi-honest
SecureNN [WGC19]	3-party	crypto	T	semi-honest
Private retrieval and nearest-neighbour search				
Sanns [CCD+20] [CCD+20]	client–server	crypto	S	semi-honest
Tiptoe [HDCZ23] [HDCZ23]	single server	crypto	S	semi-honest
Pacmann [ZSF25] [ZSF25]	single server	crypto	S	semi-honest
Compass [ZPZP25] [ZPZP25]	single server	crypto	S	compromised svr.
Panther [LHZ+25] [LHZ+25]	single server	crypto	S	semi-honest
Servan-Schreiber et al. [SLD22]†	2 non-colluding	crypto	S	semi-honest
Wally [ABG+24] [ABG+24]	server-side	DP	S	semi-honest
Asharov et al. [AHLR18]	2-party	crypto	S	semi-honest
Private information retrieval				
Chor et al. [CGKS95]†	multi-server n.c.	crypto	D	semi-honest
Kushilevitz–Ostrovsky [KO97]†	single server	crypto	D	semi-honest
Hafiz–Henry [HH19]	multi-server, no pair	crypto	D	1 malicious
SealPIR [ACLS18]	single server	crypto	D	semi-honest
Corrigan-Gibbs–Kogan [CK20]	single server	crypto	D	semi-honest
SimplePIR [HHC+23]	single server	crypto	D	semi-honest
Spiral [MW22]	single server	crypto	D	semi-honest
Oblivious memory				
Path ORAM [SvDS+13]†	client–server	crypto	—	semi-honest
Lu–Ostrovsky [LO13]	2-party	crypto	—	semi-honest
Floram [DS17] [DS17]	2-party	crypto	—	semi-honest
Jarecki–Wei [JW18]	3-party, 1 fault	crypto	—	semi-honest
Hamlin–Varia [HV21]	2-server	crypto	—	semi-honest
Bunn et al. [BKKO20]†	3-party	crypto	—	semi-honest
Duoram [VHG23]	2 or 3, 1 passive	crypto	—	semi-honest
Prac [SVG24]	3, none collude	crypto	—	semi-honest

* FedCF [AIK+19] offers no formal privacy notion. It relies instead on raw data not leaving the client. See Section 5.4.

Semi-honest is the norm at scale. Every recommendation system in the table is semi-honest. The exceptions elsewhere are narrow: Hafiz and Henry's PIR is computationally 1-private against a malicious server [HH19], Compass [ZPZP25] states its guarantee holds "even if the server is compromised", and Pika [Wag22] extends FSS-based function evaluation to malicious security in the honest-majority setting. Section 9.1 returns to why this is difficult.

4 Cryptographic Building Blocks

This section introduces the primitives the systems in Sections 5 and 6 are assembled from. The treatment is deliberately shallow on constructions and deliberate about *costs*, because it is the cost profile of each primitive, and in particular whether it costs communication, computation, or rounds, that determines which systems are built from which pieces.

4.1 Secret sharing

Secret sharing splits a value among parties so that authorised subsets can reconstruct it and unauthorised subsets learn nothing [Sha79]. Two variants recur throughout this survey.

In *additive* sharing over a ring $\mathcal{R} = \mathbb{Z}_{2^b}$, a value x is split as $x = x_0 + x_1 \bmod 2^b$, with one summand given to each of two parties. Addition of shared values, and multiplication by a public scalar, are then local operations requiring no communication. Multiplication of two shared values is not, and is classically handled with precomputed correlated randomness in the form of multiplication triples [Bea91].

In *replicated* 2-out-of-3 sharing, $x = x_0 + x_1 + x_2$ and party i holds the pair (x_i, x_{i+1}). Any two parties can reconstruct it, while a single party learns nothing. This is the sharing used by the three-party systems in Section 5.3, and its attraction is that a party holding two of the three summands can compute a share of a product locally, so that degree-two functions become very cheap [AFL+16].

Several systems additionally assume the *PRF model*, in which each pair of parties holds a shared pseudorandom-function key and can therefore generate correlated randomness, notably shares of zero, without communicating at all.

4.2 Garbled circuits

Garbled circuits [Yao86] allow two parties to evaluate a Boolean circuit on private inputs in a constant number of rounds. The cost is borne in bandwidth: the garbled circuit must be transmitted, so communication scales with circuit size. For data-oblivious algorithms whose circuit size grows with the whole input, as is the case for gradient descent over a rating matrix, this is the binding constraint, and it is the reason the systems in Section 5.1 are superseded by later secret-sharing approaches.

4.3 Homomorphic encryption

Homomorphic encryption permits computation directly on ciphertexts. Partially homomorphic schemes support one operation. Additively homomorphic schemes such as Paillier's [Pai99] are used in several of the systems below, while fully homomorphic encryption supports arbitrary circuits. Gentry [Gen09] established the latter is possible via *bootstrapping*: it suffices to construct a scheme that can homomorphically evaluate a slightly augmented version of its own decryption circuit.

The relevance of homomorphic encryption to this survey is structural rather than technical. It is the primitive that permits *single-server* constructions, and therefore the one that removes the non-collusion assumption discussed in Section 3.1, at a computational cost that Sections 4.5 and 6 quantify.

4.4 Function secret sharing and distributed point functions

Function secret sharing (FSS) shares a *function* rather than a value. Following Boyle et al. [BGI16], an m-party FSS scheme splits a function $f : \{0,1\}^n \to G$, for an abelian group G, into keys $k_1, \ldots, k_m$ such that $f = f_1 + \cdots + f_m$ and every strict subset of the keys hides f. A *distributed point function* (DPF) is the special case in which f is a point function: non-zero at a single input and zero everywhere else [GI14].

The applications are named directly by the authors: FSS schemes are useful for "privately reading from or writing to distributed databases while minimizing the amount of communication", including "different flavors of private information retrieval (PIR), as well as a recent application of DPF for large-scale anonymous messaging" [BGI16]. The same work reports reducing the key size of the earlier PRG-based DPF construction "roughly by a factor of 4" [BGI16].

The property that matters for everything downstream is *compression*. A DPF key is logarithmic in the domain size, so a party can hold a short key and expand it into a long pseudorandom vector that differs from its counterpart at exactly one secret position. This is what makes the read layer of Section 4.5 and the DORAM of Section 4.6 practical.

FSS also generalises beyond point functions into a technique for secure computation. Boyle et al. [BCG+21] evaluate a gate g using an FSS scheme for the offset family $g_r(x) = g(x + r)$, giving schemes for "zero test, integer comparison, ReLU, and spline functions" and reporting "significant savings in online communication and round complexity compared to alternative techniques based on garbled circuits or secret sharing", along with "roughly 4× reduction in key size" for the distributed comparison function. Subsequent work builds function libraries on this foundation: LLAMA [GKCG22] operates in the trusted dealer model, "where a dealer provides input independent correlated randomness to both parties", yielding "a very lightweight online phase", while noting that prior FSS-based two-party work lacked "support for math functions (e.g., sigmoid, and reciprocal square root)" and restricted all values to a single bitwidth. Pika [Wag22] reports protocols for division, exponentiation, logarithm and tanh with "constant round complexity (3 for semi-honest, 4 for malicious)" and communication "54–121× lower than prior art", and, unusually for this survey, extends them to malicious security in the honest-majority setting. Grotto [SVLH23] contributes a structural observation about the DPF tree that lets "a single DPF" do what state-of-the-art approaches "require many DCFs to do".

4.5 Private information retrieval

Private information retrieval (PIR) lets a client fetch record j from a server-held database without the server learning j. The literature divides on the trust assumption.

Multi-server PIR distributes the database across servers assumed not to collude [CGKS95]. Hafiz and Henry [HH19] study this setting, describing "several untrusted servers work to obliviously service remote clients' requests for data and yet no pair of servers colludes", and report efficiency "with respect to every cost metric — download, upload, computation, and round complexity". Their family of protocols recovers, as special cases, the two-server instances of the seminal scheme of Chor et al. and the DPF-based construction of Boyle et al. [HH19]. This is the scheme Pirsona [VBH21] builds on.

Single-server PIR removes the non-collusion assumption at a computational price [KO97]. SimplePIR [HHC+23] is presented as "the fastest single-server private information retrieval scheme known to date", with security under learning-with-errors, reporting "10 GB/s/core server throughput" that "approaches ... the performance of the fastest two-server private-information-retrieval schemes (which require non-colluding servers)". The authors state the cost plainly: "relatively large communication costs: to make queries to a 1 GB database, the client must download a 121 MB 'hint' about the database contents", after which each query needs 242 KB. A variant, DoublePIR, shrinks the hint to 16 MB at 345 KB per query and 7.4 GB/s/core [HHC+23]. Spiral [MW22] composes two lattice-based encryption schemes and reports "at least a 4.5× reduction in query size, 1.5× reduction in response size, and 2× increase in server throughput compared to previous systems".

Two further directions matter later. Corrigan-Gibbs and Kogan [CK20] construct single-server schemes with "sublinear amortized server time" by having the client "first fetch a small 'hint' about the database contents", paying a linear cost once and answering subsequent queries in sublinear time. This preprocessing paradigm is what Pacmann (Section 6.3) depends on. Angel et al. [ACLS18] reduce query size "up to 274×" and introduce probabilistic batch codes giving "up to 40× speedup over processing queries one at a time", which matters for any recommender serving many users concurrently.

4.6 Oblivious RAM and distributed ORAM

Oblivious RAM hides *access patterns*, not merely contents [GO96]. Path ORAM [SvDS+13] is the canonical tree-based construction, "an extremely simple Oblivious RAM protocol with a small amount of client storage" achieving $O(\log N)$ bandwidth for blocks of size $\Omega(\log^2 N)$ bits.

Classical ORAM assumes a client with plaintext visibility talking to an untrusted server. In secure computation there is no such split: the computing parties are both client and server, and no party may learn the access pattern. This is *distributed* ORAM, introduced for two-party computation by Lu and Ostrovsky [LO13] as an alternative to unrolling a RAM program into a circuit.

Floram [DS17] made DORAM practical by building it from FSS. The authors report improving prior constructions' "access time by a factor of up to ten, their memory overhead by a factor of one hundred or more, and their initialization time by a factor of thousands", and instantiating ORAMs holding 2^{34} bytes. They are explicit about the trade they make: deriving the construction from FSS "significantly reduces the amount of secure computation required to implement an ORAM access, albeit at the cost of $O(n)$ efficient local memory operations" [DS17]. That trade, accepting linear local computation to buy sublinear communication, recurs throughout Section 6.

Duoram [VHG23] follows Floram's DPF-based approach and reports reducing communication further: for memories with n addressable locations it performs "a sequence of m arbitrarily interleaved reads and writes" using $O(m \lg n)$ words against Floram's $O(m\sqrt{n})$, with most of the work moved into "a data-independent preprocessing phase, leaving just $O(m)$ words of online communication cost for the sequence — i.e., a constant online communication cost per memory access". Other points in the design space include two-server constructions achieving "sublinear server computation and constant rounds" [HV21], and three-party protocols matching client-server Path ORAM in round complexity [JW18, BKKO20].

Oblivious *data structures* build on this: Wang et al. [WNL+14] apply pointer-based and locality-based techniques to "maps, sets, priority-queues, stacks, deques", and Prac [SVG24] contributes binary search, heap and AVL-tree protocols over Duoram.

4.7 Fixed-point arithmetic, and why it dominates the cost

The primitives above compute over rings and finite fields, while recommendation computes over the reals. Bridging that gap with fixed-point arithmetic is where much of the practical cost of these systems is incurred, which is why it deserves its own subsection rather than a footnote.

A real value v is represented as $\lfloor v \cdot 2^t \rfloor \in \mathcal{R}$. Addition is then free, but the product of two t-scaled values is $2t$-scaled and must be *truncated* back. Repeated multiplication also requires *normalisation* to stop values growing without bound. Both are non-linear operations, and both are expensive under secret sharing.

Consequently a recurring theme in Section 5 is that the linear algebra is cheap and the non-linear steps are not. SecureML [MZ17] contributes "new techniques to support secure arithmetic operations on shared decimal numbers" and "MPC-friendly alternatives to non-linear functions such as sigmoid and softmax". Escudero et al. [EGK+20] introduce *extended doubly-authenticated bits*, "shared integers in the arithmetic domain whose bit decomposition is shared in the binary domain", used to "considerably increase the efficiency of non-linear operations such as truncation, secure comparison and bit-decomposition". General-purpose frameworks make the same division visible: ABY [DSZ15] and its successor [PSSY21] are *mixed-protocol* systems precisely because different operations are cheapest under different sharings, and MP-SPDZ [Kel20] implements "34 MPC protocol variants" behind one interface to make such comparisons possible.

5 Private Training

Every system in this section computes the same object: a low-rank factorisation $\mathbf{U} \approx \mathbf{A} \cdot \mathbf{B}$ of a rating matrix nobody is allowed to see. They differ in the cryptographic family they use and, as this section argues, in the *algorithm* they choose to run, a choice that turns out to be driven by cryptographic rather than statistical considerations.

5.1 The garbled-circuit era

The first practical system was that of Nikolaenko et al. [NIW+13], which evaluates gradient descent inside a garbled circuit across two non-colluding servers.[2] Its cost profile is what one expects from Section 4.2: because the computation must be data-oblivious, the circuit is sized by the whole rating matrix, and communication follows. Nudge reports this family requiring over four orders of magnitude more communication and computation than secret-sharing approaches [HDCB26].

The broader lesson is architectural rather than about any one paper. A circuit-based approach pays for every operation in circuit size, so an algorithm with many cheap linear steps is no cheaper than one with few expensive ones. Secret sharing inverts that relationship, and the systems below exploit the inversion.

5.2 Homomorphic encryption

Homomorphic approaches remove the non-collusion assumption by keeping the data encrypted at a single server (Section 4.3). The trade is computational: fully homomorphic evaluation of a training loop is expensive, and the literature accordingly tends toward partially homomorphic schemes used for specific sub-operations rather than end-to-end training. FedMF's use of homomorphic encryption

[2] We were unable to obtain this paper. The characterisation here follows Nudge [HDCB26]'s description and measurements of it.

to protect gradient uploads [CWCY21] is the pattern that recurs: homomorphic encryption applied surgically to the one channel that leaks, rather than to the whole computation.

5.3 Secret-sharing MPC

This is the line along which the field has advanced furthest, and it is worth following the cost argument rather than just the chronology.

SecureML [MZ17] established the two-server pattern: data owners "distribute their private data among two non-colluding servers who train various models on the joint data using secure two-party computation". Its contributions are revealing of where the difficulty lies: "new techniques to support secure arithmetic operations on shared decimal numbers" and "MPC-friendly alternatives to non-linear functions such as sigmoid and softmax". The linear algebra was not the problem. Fixed-point arithmetic and non-linearities were.

SecureNN [WGC19] moved to three parties, giving protocols for "matrix multiplication, convolutions, Rectified Linear Units, Maxpool, normalization and so on . . . such that no single party learns any information about the data", and reports secure inference outperforming prior two- and three-server work by 6×–113×. Again the enumerated contributions are the non-linear layers.

Pirsona [VBH21] applied this family to recommendation directly, combining collaborative filtering with private information retrieval. Its training component is a "carefully optimized 4PC Boolean matrix factorization", and the authors are candid about the price: they "opted for the most performant primitives available (at the expense of rather strong non-collusion assumptions)" [VBH21]. Because its delivery mechanism is equally central, it is discussed in full in Section 7.

Nudge [HDCB26] is the current state of the art, and its central move is to change the algorithm. Under 2-out-of-3 replicated sharing a shared matrix–vector product can be computed with very little communication, so an algorithm consisting mostly of matrix–vector products separated by a few non-linear steps is far cheaper than one that is not. Nudge therefore replaces gradient descent with *power iteration*, leaving truncation and normalisation as the only operations that cost rounds. The user embeddings $\mathbf{A}$ remain secret-shared while the item embeddings $\mathbf{B}$ are revealed in the clear, a design decision with consequences taken up in Sections 7 and 9.4.

The reported results give the field its current reference point: on Netflix, "half a million users and ten thousand items", running "on three 192-core servers on a local-area network", Nudge "privately learns a recommender model in 50 mins with 40 GB of server-to-server communication", scoring "0.29 out of 1.0" on nDCG@20, "on par with non-private matrix factorization and just shy of non-private neural recommenders, which score 0.31" [HDCB26]. Privacy holds "against an adversary that compromises the entire secret state of one server".

That quality figure deserves emphasis. It is the strongest available evidence that cryptographic privacy in recommendation need not cost accuracy: the cost is paid in compute and communication, not in the quality of what users are recommended.

5.4 Federated approaches

Federated learning keeps raw data on the client and uploads only model updates. Ammad-ud-din et al. [AIK+19] report "the first federated implementation of a Collaborative Filter", with updates "based on a stochastic gradient approach", and find that "a collaborative filter can be federated without a loss of accuracy compared to a standard implementation".

The privacy of that arrangement is weaker than it appears, and the clearest evidence comes from inside the federated literature itself. Chai et al. [CWCY21] design a user-level distributed

matrix factorisation in which each user "only uploads the gradient information (instead of the raw preference data) to the server", and then explicitly "prove that it could still leak users' raw data". Their response is to add homomorphic encryption to the gradient channel.

This is the cleanest illustration of the distinction drawn in Section 3.2. Federated learning is an *architecture*, not a privacy notion: it changes where computation happens without providing a guarantee about what the resulting messages reveal. Where a federated system does provide a guarantee, it is because a cryptographic or differentially private mechanism was added on top.

5.5 Differential privacy

McSherry and Mironov [MM09] brought differential privacy to recommendation in the context of the Netflix Prize. The guarantee is orthogonal to everything above: it bounds what the *released model* reveals about any individual, rather than protecting the computation. It is therefore complementary to the cryptographic approaches. As Section 9.4 argues, it addresses a leak that cryptography structurally cannot.

5.6 Where each family stops

Garbled circuits stop at scale. Circuit size grows with the data, and communication with it.

Homomorphic encryption stops at cost. It buys the single-server trust model, and pays in computation. In practice it is applied to specific channels rather than whole training loops.

Secret-sharing MPC stops at the trust assumption. It is now fast enough for real datasets, but every system in this family requires two or three non-colluding parties, and all of them are semi-honest.

Federated learning stops at the guarantee. Without an added cryptographic or DP mechanism, gradient uploads leak [CWCY21].

Differential privacy stops at utility, and at scope: it protects the output, not the computation.

There is also a limit none of these families addresses, because it is not a training problem at all. Every system in this section ends with a model, or with scores. None of them delivers the recommended *item* to the user. That is the subject of Section 6.

6 Private Serving and Retrieval

Section 5 ended with a trained model. Two problems remain: computing a ranking for a user without revealing their profile, and fetching the recommended item without revealing which item it was. Both reduce to retrieval, and both have been studied largely by a different community, and mostly under the name *private nearest-neighbour search* rather than recommendation.

6.1 The obstruction: indices are data-dependent

Retrieval in the clear is fast because of indices: a k-d tree, an inverted file, or a navigable-graph structure lets a query touch a small fraction of the corpus. An index is useful precisely *because* the path taken through it depends on the query, and that is exactly what obliviousness forbids, since the access pattern would reveal the query.

This gives the field its organising trade-off. Either scan the whole corpus for every query, and accept cost linear in corpus size, or keep the index and hide the traversal, which requires oblivious memory (Section 4.6) and its attendant overhead. Every system below takes a position on this trade.

6.2 Linear-scan approaches

Sanns [CCD+20] [CCD+20] is the origin point, and it names our application directly: secure k-NN search is "the backbone of several cloud-based services such as recommender systems, face recognition, and database search". It offers two protocols: "an optimized linear scan and a protocol based on a novel sublinear time clustering-based algorithm", proved secure "in the standard semi-honest model" and built from "lattice-based additively homomorphic encryption, distributed oblivious RAM, and garbled circuits". Notably for Section 7, it contributes "a new circuit for the approximate top-k selection from n numbers that is built from $O(n + k^2)$ comparators": top-k selection is itself a protocol-design problem.

Tiptoe [HDCZ23] [HDCZ23] is the largest-scale instance of the linear approach. It reduces "private full-text search to private nearest-neighbor search" using semantic embeddings, then implements the latter with linearly homomorphic encryption. Its trust model is the strongest in this section: "Tiptoe's privacy guarantee is based on cryptography alone; it does not require hardware enclaves or non-colluding servers." On a 45-server cluster searching 360 million web pages it reports "145 core-seconds of server compute, 56.9 MiB of client-server communication (74% of which occurs before the client enters its search query), and 2.7 seconds of end-to-end latency".

Its quality reporting is a model of the norms discussed in Section 8. On MS MARCO, Tiptoe "ranks the best-matching result in position 7.7 on average", which the authors state is "worse than a state-of-the-art, non-private neural search algorithm (average rank: 2.3), but is close to the classical tf-idf algorithm (average rank: 6.7)".

6.3 Sublinear and index-based approaches

Pacmann [ZSF25] [ZSF25] takes the opposite position on the trade-off, and does so by moving work to the client. "Unlike prior constructions that run encrypted search on the server side, Pacmann carefully offloads limited computation and storage to the client, no longer requiring computationally-intensive cryptographic techniques." The client runs a graph-based ANN search "where in each hop on the graph, the client privately retrieves local graph information from the server", combining a graph ANN algorithm with "a recent class of PIR schemes that trade offline preprocessing for online computational efficiency", the preprocessing paradigm of Corrigan-Gibbs and Kogan [CK20]. It reports "up to 2.5× better search accuracy on real-world datasets than prior work", reaching "90% quality of a state-of-the-art non-private ANN algorithm", and on datasets up to 100 million vectors "up to 62% reduction in computation time and 22% reduction in overall latency".

Compass [ZPZP25] [ZPZP25] keeps the index server-side and hides the traversal with oblivious memory, contributing "a novel way to traverse a state-of-the-art graph-based semantic search index and a white-box co-design with Oblivious RAM". Its named techniques are Directional Neighbor Filtering, Speculative Neighbor Prefetch, and Graph-Traversal Tailored ORAM, all consequences of that co-design, and it reports "user-perceived latencies within or around a second", "orders of magnitude faster than baselines under various network conditions". It also states the strongest adversary model in this section: privacy of "data, queries, and results, even if the server is compromised".

6.4 Relaxing the guarantee

Panther [LHZ+25] [LHZ+25] works in the single-server setting and characterises the field's prior trade-off precisely: earlier work "either suffer[s] from high communication cost (Chen et al., USENIX Security 2020) or work[s] under a stronger security assumption of two non-colluding servers (Servan-Schreiber et al., SP 2022)". Through "co-designs of private information retrieval, secret-sharing, garbled circuits, and homomorphic encryption" it reports answering "an ANNS query on 10 million points in 18 seconds with 284 MB of communication", "more than 7.8× faster and 20× more compact" than the former.

Wally [ABG+24] [ABG+24] relaxes the privacy notion rather than the trust model, using "differential privacy to break the linear computation barrier of fully-oblivious schemes". Its framing of the prior art is a useful summary of this whole section: "In prior systems like Tiptoe, the server must process the entire database per query to hide the access pattern, resulting in low throughput (909 QPS) and high communication (17.4 MB) on a 3.2-million-entry database. Sublinear alternatives like Pacmann avoid full scans but require 614 MB of client storage and an offline phase where clients stream the entire database." Its own approach batches queries from many non-coordinating clients, each adding "a few fake queries" to obscure which cluster it wants.

Wally sits between two of the axes of Section 3: its trust model is conventional, but its privacy notion is differential rather than cryptographic. That a serious system occupies this position is the clearest justification for keeping trust model and privacy notion as separate axes.

6.5 When the functionality itself is sensitive

Asharov et al. [AHLR18] study similar-patient queries over genomic data, where "a doctor that holds the genome of his/her patient may try to find other individuals with 'close' genomic data". They report an approximation that "returns the exact closest records in 98% of the queries" with a protocol running "in just a few seconds . . . on databases with thousands of records".

The structural point transfers directly to recommendation. Both settings involve a query that is sensitive, a corpus that is sensitive, and, critically, an *answer* that is sensitive. Protecting the computation does not protect the answer, a theme Section 9.4 develops.

6.6 Where each approach stops

Linear scan stops at throughput. Touching the whole corpus per query bounds queries per second, and Wally's measurements of Tiptoe quantify the bound [ABG+24].

Client-side graph traversal stops at client resources. Pacmann's design deliberately shifts cost to the client, and Wally reports the storage and offline streaming that this requires [ABG+24].

ORAM-based traversal stops at the cost of obliviousness itself, which is why Compass needed a white-box co-design rather than an off-the-shelf ORAM.

DP-based relaxation stops at the guarantee: what is obtained is a differential-privacy statement, not a cryptographic one.

One further limitation applies to the whole section and is easy to miss. Every system here answers a *query*. None of them is connected to a privately trained model: the corpus is given, the embeddings are given, and where they come from is out of scope. Section 7 takes up what happens when the two halves are put together.

7 Systems Spanning Multiple Stages

Sections 5 and 6 surveyed two bodies of work that solve adjacent halves of the same problem. This section asks which systems attempt more than one stage of the pipeline at once, and what is learned from the attempt.

7.1 Pirsona: collection, training and delivery together

Pirsona [VBH21] is the system in this survey that spans the most stages. Its stated goal is the construction of "practically efficient e-commerce and digital media delivery systems that can provide personalized content recommendations based on their users' historical consumption patterns while simultaneously keeping said consumption patterns private" [VBH21].

The authors frame the difficulty as one of apparently incompatible goals: private information retrieval and collaborative filtering are "seemingly antithetical primitives". PIR exists to hide which item a user fetches. Collaborative filtering works by relating users to one another through exactly what they fetched.

The resolution has three parts. Delivery uses the computationally 1-private multiserver PIR of Hafiz and Henry [HH19]. Collection is folded into delivery: because a PIR query already encodes the requested item in shared form, the servers extract secret-shared consumption histories from the query traffic itself, so no separate ratings upload is required. Training then runs a "carefully optimized 4PC Boolean matrix factorization" over those shared histories.

Two aspects are worth carrying forward. First, the collection-from-delivery mechanism is, as far as this survey found, unique to Pirsona: it makes the pipeline a cycle rather than a sequence. Second, the authors are explicit that the design "opted for the most performant primitives available (at the expense of rather strong non-collusion assumptions)": a s+1-server, pairwise non-colluding model.

7.2 Nudge: collection, training and serving

Nudge [HDCB26] spans collection, training and serving. Users secret-share ratings, three servers train by power iteration, and the servers then compute each user's recommendation scores in shared form and forward them, so the servers never learn the ranking.

What it does not do is stated by its authors as a non-goal: Nudge "relies on other means (e.g., Apple's private relay, Tor, or cryptographic private information retrieval) to let users fetch data items in a private way" [HDCB26]. The pipeline is protected up to the point where the user acts on the recommendation.

7.3 The composition problem

Putting the two halves together is not simply a matter of running one after the other, for three reasons that this survey can identify from the sources read.

Trust models must agree. A composed system inherits the union of its components' assumptions. Here the two lines happen to be compatible: Nudge assumes three servers with at most one compromised [HDCB26], Duoram tolerates "a single passive corruption" in a two- or three-party setting [VHG23], and Prac assumes "none of the three parties . . . collude" [SVG24]. But that is a fact to be checked, not assumed. Where a retrieval system is single-server (Tiptoe, Panther [LHZ+25]) and a training system requires non-collusion, the composition is governed by the weaker assumption.

Party roles need not match. The three-party systems in this survey assign different roles to their third party. In Nudge all three servers hold shares of the input. In Prac, P_0 and P_1 hold the data while P_2 holds no input, supplying correlated randomness and participating in the memory protocols. In the (2+1)-party model used by Grotto, the third party is an offline dealer that supplies correlated randomness and does not participate online. Composing systems that assume different roles for a third party is not a matter of running them side by side.

The interface between stages carries information. Nudge reveals the item embeddings $\mathbf{B}$ in the clear as a deliberate design choice, and forwards score vectors to users. Whatever the delivery mechanism, it operates on outputs that are themselves informative, a point Section 9.4 develops.

7.4 What the field has and has not built

Reading Table 1 by column rather than by row: the training stage is well served, the serving stage is well served, and *delivery* is addressed almost entirely by the PIR literature in isolation from any recommender.

The private-retrieval systems of Section 6 are, with the exception of Pirsona, not connected to privately trained models. They take a corpus and embeddings as given. The private-training systems of Section 5 produce models and scores and stop. Pirsona spans both, but predates the private-nearest-neighbour line almost in its entirety: Sanns appeared in 2020, Tiptoe in 2023, and Pacmann, Compass, Panther and Wally in 2024–2025, against Pirsona's publication in 2021.

This survey does not claim that composing these lines is straightforward, nor that no unpublished work has done so. It observes that the systems surveyed here occupy the two halves separately, that the one system spanning both uses a training core the field has since moved past, and that the current state of the art in training explicitly delegates the remaining stage. That observation is the substance of Section 9.3.

8 How This Literature Evaluates Itself

The numbers quoted in Sections 5 and 6 are not directly comparable, and this section explains why. It is included because a reader who takes those figures at face value will draw wrong conclusions, and because the norms described here are what any new work in the area will be measured against.

8.1 Datasets

The recommendation systems in this survey evaluate on public rating datasets, principally MovieLens [HK16] and the Netflix Prize data. Nudge [HDCB26] reports at Netflix scale ("half a million users and ten thousand items"), while the systems it compares against report on subsets of MovieLens-100K. Scale differences of this magnitude are not a detail: a protocol whose cost grows with the product of users and items behaves qualitatively differently at the two ends of that range.

The private-retrieval systems evaluate on document and vector corpora instead: Tiptoe [HDCZ23] on 360 million web pages with MS MARCO for quality, Panther [LHZ+25] on "10 million points", Pacmann [ZSF25] on datasets "up to 100 million vectors". There is no shared benchmark spanning the two halves of the pipeline, which is a direct consequence of the split described in Section 7.

8.2 Quality metrics

Ranking quality is reported with nDCG@k and recall@k (Section 2.4). The important methodological point is that quality must be reported *at all*: a private recommender that degrades recommendations

has moved cost from one column to another rather than eliminating it.

Two systems in this survey report quality against a non-private baseline in a way worth emulating. Nudge states nDCG@20 of "0.29 out of 1.0 . . . on par with non-private matrix factorization and just shy of non-private neural recommenders, which score 0.31" [HDCB26]. Tiptoe states its average rank of 7.7 is "worse than a state-of-the-art, non-private neural search algorithm (average rank: 2.3), but is close to the classical tf-idf algorithm (average rank: 6.7)" [HDCZ23]. In both cases the unflattering comparison is given alongside the favourable one.

8.3 Cost metrics and the network model

Three quantities matter, and they trade against each other: local computation, bandwidth, and round complexity.

Which of them dominates depends entirely on the deployment. Floram [DS17]'s authors accept "$O(n)$ efficient local memory operations" in exchange for less secure computation [DS17], a trade that is favourable on a fast machine and unfavourable on a slow link. Duoram moves work into "a data-independent preprocessing phase, leaving just $O(m)$ words of online communication" [VHG23]. This is an offline/online split that is invisible in a single end-to-end number.

Because of this, network conditions are part of the result rather than a detail of the setup. Nudge's headline figure is a local-area network measurement [HDCB26]. ABY2.0 benchmarks "over LAN and WAN networks" [PSSY21]. Compass reports being "orders of magnitude faster than baselines under various network conditions" [ZPZP25]. A figure quoted without its network model, hardware and dataset is not a result, and this survey has tried to quote none.

8.4 Baselines and artifacts

General-purpose MPC frameworks serve as the common reference point. MP-SPDZ implements "34 MPC protocol variants . . . with the same high-level programming interface", explicitly so that it "considerably simplifies comparing the cost of different protocols and security models" [Kel20]. ABY [DSZ15] and ABY2.0 [PSSY21] play a similar role for mixed-protocol two-party computation.

The strongest papers in this survey also compare against the honest, unflattering baseline. Both of the quality comparisons quoted in Section 8.2 are of that kind, and Wally's characterisation of its predecessors, reporting Tiptoe at "909 QPS" and Pacmann at "614 MB of client storage" [ABG+24], is a further example of a paper stating precisely what it improves on.

8.5 Why cross-paper comparison is unsound

Collecting the above: systems in this survey differ in dataset and scale, in hardware, in network model, in whether preprocessing is counted, in whether an adversary is semi-honest or malicious, in how many non-colluding parties are assumed, and in which pipeline stage is measured at all. Any two numbers drawn from different papers differ along several of these axes simultaneously.

This is not a criticism of the individual papers, which are generally careful about their own setup. It is a statement about what a *survey* can responsibly do: report each system's claims in the terms its authors stated them, and decline to construct a league table. That is the approach taken throughout Sections 5 and 6, and the absence of a shared benchmark is recorded as an open problem in Section 9.5.

9 Open Problems

The problems below are drawn from two sources: limitations the surveyed authors state about their own work, and structural gaps visible in Table 1. They are not a wish list.

9.1 Malicious security at scale

Every recommendation system in Table 1 is semi-honest, and so is nearly every system evaluated at realistic scale. Nudge [HDCB26] lists providing malicious security as the first of its three explicitly stated open questions.

The exceptions show the cost. Hafiz and Henry achieve computational 1-privacy against a malicious server, but in the multi-server PIR setting rather than for training [HH19]. Pika extends FSS-based function evaluation to "malicious adversaries in the honest majority setting", reporting round complexity rising from 3 to 4 [Wag22]. Compass [ZPZP25] claims privacy "even if the server is compromised". Each of these is narrower than end-to-end malicious security for a full recommendation pipeline, which no surveyed system provides.

9.2 Reducing the non-collusion assumption

The fastest protocols in every family assume non-colluding parties, and the assumption is about organisations rather than mathematics. Nudge states the goal directly among its open questions: serving private recommendations with only *two* non-colluding parties.

The single-server line shows the trade rather than resolving it. SimplePIR removes the assumption entirely and reports throughput approaching that of two-server schemes, at the cost of a 121 MB client hint per gigabyte of database [HHC+23]. Tiptoe likewise requires "no hardware enclaves or non-colluding servers" and pays for it in server compute [HDCZ23]. Whether a recommendation pipeline can be built in the single-server model at competitive cost is open.

9.3 The delivery stage

This is the gap most visible in Table 1. Delivery is addressed almost entirely by the PIR literature, in isolation from any recommender. Pirsona [VBH21] is the one surveyed system connecting delivery to training, and it predates the private-nearest-neighbour work of Section 6 almost entirely.

The current state of the art in training states the gap in its own non-goals: Nudge "relies on other means (e.g., Apple's private relay, Tor, or cryptographic private information retrieval) to let users fetch data items in a private way" [HDCB26]. Meanwhile the retrieval systems of Section 6 take their corpora and embeddings as given.

Two questions follow, neither of which this survey found answered. What does it cost to compose a modern private-retrieval backend with a privately trained recommender, given the composition obstacles of Section 7.3? And does Pirsona's collection-from-delivery mechanism, harvesting shared consumption histories from PIR query traffic, transfer to a modern training substrate?

9.4 Leakage through the recommendation itself

This is the deepest problem in the section, and it is not a protocol failure.

Cryptographic privacy, as defined in Section 3.2, guarantees that nothing leaks *beyond the agreed output*. It says nothing about what the output itself reveals. A recommender's output is a statement

about the user, delivered to a system that observes whether the user acts on it. Calandrino et al. [CKN+11] address precisely the privacy risks of collaborative filtering in this sense.[3] Asharov et al. [AHLR18] encounter a structurally identical situation in genomic similar-patient search, where the answer to a legitimate query is itself disclosive.

Nudge's design makes the point concrete: it reveals the item embeddings $\mathbf{B}$ in the clear, and reveals recommendation scores to the user, both by design [HDCB26]. Neither is a flaw: both are what the system is for. But both are information that no amount of additional cryptography inside the protocol would remove.

Differential privacy addresses exactly this class of leak, at a utility cost [DR14, MM09]. Composing cryptographic and differentially private guarantees in a recommendation pipeline, protecting the computation with one and the output with the other, appears only partially explored. Wally [ABG+24] is the system in this survey that most clearly combines the two notions, and it does so for retrieval rather than for recommendation.

9.5 Evaluation and comparability

Section 8 argued that published numbers in this field are not comparable across papers, because they differ simultaneously in dataset, scale, hardware, network model, adversary model, trust assumption and pipeline stage. There is no shared benchmark, and in particular none spanning both halves of the pipeline.

This is an unglamorous problem, but it is tractable, and its absence has a measurable cost: every paper must re-implement its predecessors to make a comparison, which is precisely what Nudge reports doing for its baselines [HDCB26].

9.6 The direction we intend to pursue

The course project following this survey engages with Section 9.3: the composition of a privately trained recommender with a private delivery mechanism. We make no claim here about the difficulty or the outcome of that work.

10 Conclusion

Privacy-preserving recommendation is not one problem but four, corresponding to the stages of the pipeline in Figure 1. Reading the literature through that lens is what this survey has tried to make possible, and it explains why systems that appear to be alternatives frequently are not.

Two of those stages are in good shape. Private training has advanced to the point where cryptographic privacy costs compute and communication rather than recommendation quality: Nudge reports nDCG@20 on par with non-private matrix factorisation [HDCB26]. Private retrieval has developed rapidly and independently, from secure k-nearest-neighbour search through web-scale private search to index-based sublinear schemes. The delivery stage is served by a mature private-information-retrieval literature that is, with one exception, not connected to any recommender.

That exception, Pirsona [VBH21], is also the reason the gap is worth naming: it demonstrates that collection, training and delivery can be addressed together, and its training core has since been superseded while the retrieval literature it would now draw on did not yet exist. The current state of the art in training, meanwhile, states the remaining stage as a non-goal [HDCB26].

[3] As noted in Section 2.3, we could not obtain this paper and cite it only for the claim its title makes.

Two caveats bound what this survey establishes. It reports each system in the terms its authors used and declines to rank them, because Section 8 argues that cross-paper comparison in this field is unsound. And its claims about individual works are bounded by what we read: for papers we could not obtain, we have said so at the point of use rather than paraphrasing them.

The problems collected in Section 9, namely malicious security at scale, weakening the non-collusion assumption, connecting delivery to training, and the leakage that survives any protocol because it is carried by the recommendation itself, are the ones the surveyed authors themselves identify. The last of these is the most fundamental, since no improvement in protocol design addresses it.

References

[ABG+24] Hilal Asi, Fabian Boemer, Nicholas Genise, Muhammad Haris Mughees, Tabitha Ogilvie, Rehan Rishi, Guy N. Rothblum, Kunal Talwar, Karl Tarbe, Ruiyu Zhu, and Marco Zuliani. Scalable private search with wally. *CoRR*, abs/2406.06761, 2024.

[ACLS18] Sebastian Angel, Hao Chen, Kim Laine, and Srinath T. V. Setty. PIR with compressed queries and amortized query processing. In *2018 IEEE Symposium on Security and Privacy, SP 2018, Proceedings, 21-23 May 2018, San Francisco, California, USA*, pages 962–979. IEEE Computer Society, 2018.

[AFL+16] Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. High-throughput semi-honest secure three-party computation with an honest majority. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, pages 805–817. ACM, 2016.

[AHLR18] Gilad Asharov, Shai Halevi, Yehuda Lindell, and Tal Rabin. Privacy-preserving search of similar patients in genomic data. *Proc. Priv. Enhancing Technol.*, 2018(4):104–124, 2018.

[AIK+19] Muhammad Ammad-ud-din, Elena Ivannikova, Suleiman A. Khan, Were Oyomno, Qiang Fu, Kuan Eeik Tan, and Adrian Flanagan. Federated collaborative filtering for privacy-preserving personalized recommendation system. *CoRR*, abs/1901.09888, 2019.

[BCG+21] Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Nishant Kumar, and Mayank Rathee. Function secret sharing for mixed-mode and fixed-point secure computation. In *Advances in Cryptology - EUROCRYPT 2021*, pages 871–900. Springer, 2021.

[Bea91] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *Advances in Cryptology - CRYPTO '91, 11th Annual International Cryptology Conference, Santa Barbara, California, USA, August 11-15, 1991, Proceedings*, pages 420–432. Springer, 1991.

[BGI16] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, pages 1292–1303. ACM, 2016.

[BKKO20] Paul Bunn, Jonathan Katz, Eyal Kushilevitz, and Rafail Ostrovsky. Efficient 3-party distributed ORAM. In *Security and Cryptography for Networks - 12th International Conference, SCN 2020*, pages 215–232. Springer, 2020.

[CCD+20] Hao Chen, Ilaria Chillotti, Yihe Dong, Oxana Poburinnaya, Ilya P. Razenshteyn, and M. Sadegh Riazi. SANNS: scaling up secure approximate k-nearest neighbors search. In *29th USENIX Security Symposium, USENIX Security 2020*, pages 2111–2128. USENIX Association, 2020.

[CGKS95] Benny Chor, Oded Goldreich, Eyal Kushilevitz, and Madhu Sudan. Private information retrieval. In *36th Annual Symposium on Foundations of Computer Science, FOCS 1995*, pages 41–50. IEEE Computer Society, 1995.

[CK20] Henry Corrigan-Gibbs and Dmitry Kogan. Private information retrieval with sublinear online time. In *Advances in Cryptology - EUROCRYPT 2020*, pages 44–75. Springer, 2020.

[CKN+11] Joseph A. Calandrino, Ann Kilzer, Arvind Narayanan, Edward W. Felten, and Vitaly Shmatikov. "you might also like: " privacy risks of collaborative filtering. In *32nd IEEE Symposium on Security and Privacy, SP 2011*, pages 231–246. IEEE Computer Society, 2011.

[CWCY21] Di Chai, Leye Wang, Kai Chen, and Qiang Yang. Secure federated matrix factorization. *IEEE Intell. Syst.*, 36(5):11–20, 2021.

[DCJ19] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. Are we really making much progress? A worrying analysis of recent neural recommendation approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems, RecSys 2019*, pages 101–109. ACM, 2019.

[DR14] Cynthia Dwork and Aaron Roth. The algorithmic foundations of differential privacy. *Found. Trends Theor. Comput. Sci.*, 9(3-4):211–407, 2014.

[DS17] Jack Doerner and Abhi Shelat. Scaling ORAM for secure computation. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, pages 523–535. ACM, 2017.

[DSZ15] Daniel Demmler, Thomas Schneider, and Michael Zohner. ABY - A framework for efficient mixed-protocol secure two-party computation. In *22nd Annual Network and Distributed System Security Symposium, NDSS 2015*. The Internet Society, 2015.

[EGK+20] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In *Advances in Cryptology - CRYPTO 2020*, pages 823–852. Springer, 2020.

[EKR18] David Evans, Vladimir Kolesnikov, and Mike Rosulek. A pragmatic introduction to secure multi-party computation. *Found. Trends Priv. Secur.*, 2(2-3):70–246, 2018.

[Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing, STOC 2009*, pages 169–178. ACM, 2009.

[GI14] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In *Advances in Cryptology - EUROCRYPT 2014*, pages 640–658. Springer, 2014.

[GKCG22] Kanav Gupta, Deepak Kumaraswamy, Nishanth Chandran, and Divya Gupta. LLAMA: A low latency math library for secure inference. *Proc. Priv. Enhancing Technol.*, 2022(4):274–294, 2022.

[GO96] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious rams. *J. ACM*, 43(3):431–473, 1996.

[HDCB26] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Dan Boneh. Nudge: A private recommendations engine. In *35th USENIX Security Symposium, USENIX Security 2026*. USENIX Association, 2026. Full version: IACR ePrint 2026/179.

[HDCZ23] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Nickolai Zeldovich. Private web search with tiptoe. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023*, pages 396–416. ACM, 2023.

[HH19] Syed Mahbub Hafiz and Ryan Henry. A bit more than a bit is more than a bit better: Faster (essentially) optimal-rate many-server PIR. *Proc. Priv. Enhancing Technol.*, 2019(4):112–131, 2019.

[HHC+23] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One server for the price of two: Simple and fast single-server private information retrieval. In *32nd USENIX Security Symposium, USENIX Security 2023*, pages 3889–3905. USENIX Association, 2023.

[HK16] F. Maxwell Harper and Joseph A. Konstan. The movielens datasets: History and context. *ACM Trans. Interact. Intell. Syst.*, 5(4):19:1–19:19, 2016.

[HKV08] Yifan Hu, Yehuda Koren, and Chris Volinsky. Collaborative filtering for implicit feedback datasets. In *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM 2008*, pages 263–272. IEEE Computer Society, 2008.

[HV21] Ariel Hamlin and Mayank Varia. Two-server distributed ORAM with sublinear computation and constant rounds. In *Public-Key Cryptography - PKC 2021*, pages 499–527. Springer, 2021.

[JW18] Stanislaw Jarecki and Boyang Wei. 3pc ORAM with low latency, low bandwidth, and fast batch retrieval. In *Applied Cryptography and Network Security - 16th International Conference, ACNS 2018*, pages 360–378. Springer, 2018.

[KBV09] Yehuda Koren, Robert M. Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.

[Kel20] Marcel Keller. MP-SPDZ: A versatile framework for multi-party computation. In *CCS '20: 2020*, pages 1575–1590. ACM, 2020.

[KO97] Eyal Kushilevitz and Rafail Ostrovsky. Replication is NOT needed: SINGLE database, computationally-private information retrieval. In *38th Annual Symposium on Foundations of Computer Science, FOCS 1997*, pages 364–373. IEEE Computer Society, 1997.

[LHZ+25] Jingyu Li, Zhicong Huang, Min Zhang, Cheng Hong, Jian Liu, Tao Wei, and Wenguang Chen. Panther: Private approximate nearest neighbor search in the single server setting. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, October 13-17, 2025*, pages 365–379. ACM, 2025.

[Lin17] Yehuda Lindell. How to simulate it - A tutorial on the simulation proof technique. In *Tutorials on the Foundations of Cryptography*, pages 277–346. Springer International Publishing, 2017.

[LKHJ18] Dawen Liang, Rahul G. Krishnan, Matthew D. Hoffman, and Tony Jebara. Variational autoencoders for collaborative filtering. In *Proceedings of the 2018 World Wide Web Conference on World Wide Web, WWW 2018, Lyon, France, April 23-27, 2018*, pages 689–698. ACM, 2018.

[LO13] Steve Lu and Rafail Ostrovsky. Distributed oblivious RAM for secure two-party computation. In *Theory of Cryptography - 10th Theory of Cryptography Conference, TCC 2013*, pages 377–396. Springer, 2013.

[MM09] Frank McSherry and Ilya Mironov. Differentially private recommender systems: Building privacy into the netflix prize contenders. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Paris, France, June 28 - July 1, 2009*, pages 627–636. ACM, 2009.

[MW22] Samir Jordan Menon and David J. Wu. SPIRAL: fast, high-rate single-server PIR via FHE composition. In *43rd IEEE Symposium on Security and Privacy, SP 2022*, pages 930–947. IEEE, 2022.

[MZ17] Payman Mohassel and Yupeng Zhang. Secureml: A system for scalable privacy-preserving machine learning. In *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*, pages 19–38. IEEE Computer Society, 2017.

[NIW+13] Valeria Nikolaenko, Stratis Ioannidis, Udi Weinsberg, Marc Joye, Nina Taft, and Dan Boneh. Privacy-preserving matrix factorization. In *2013 ACM SIGSAC Conference on Computer and Communications Security, CCS'13, Berlin, Germany, November 4-8, 2013*, pages 801–812. ACM, 2013.

[NS08] Arvind Narayanan and Vitaly Shmatikov. Robust de-anonymization of large sparse datasets. In *2008 IEEE Symposium on Security and Privacy (SP 2008), 18-21 May 2008, Oakland, California, USA*, pages 111–125. IEEE Computer Society, 2008.

[Pai99] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In *Advances in Cryptology - EUROCRYPT '99, International Conference on the Theory and Application of Cryptographic Techniques, Prague, Czech Republic, May 2-6, 1999, Proceeding*, pages 223–238. Springer, 1999.

[PSSY21] Arpita Patra, Thomas Schneider, Ajith Suresh, and Hossein Yalame. ABY2.0: improved mixed-protocol secure two-party computation. In *30th USENIX Security Symposium, USENIX Security 2021*, pages 2165–2182. USENIX Association, 2021.

[Sha79] Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, 1979.

[SLD22] Sacha Servan-Schreiber, Simon Langowski, and Srinivas Devadas. Private approximate nearest neighbor search with sublinear communication. In *43rd IEEE Symposium on Security and Privacy, SP 2022*, pages 911–929. IEEE, 2022.

[ST17] Erez Shmueli and Tamir Tassa. Secure multi-party protocols for item-based collaborative filtering. In *Proceedings of the Eleventh ACM Conference on Recommender Systems, RecSys 2017*, pages 89–97. ACM, 2017.

[SvDS+13] Emil Stefanov, Marten van Dijk, Elaine Shi, Christopher W. Fletcher, Ling Ren, Xiangyao Yu, and Srinivas Devadas. Path ORAM: an extremely simple oblivious RAM protocol.

In *2013 ACM SIGSAC Conference on Computer and Communications Security, CCS'13, Berlin, Germany, November 4-8, 2013*, pages 299–310. ACM, 2013.

[SVG24] Sajin Sasy, Adithya Vadapalli, and Ian Goldberg. PRAC: round-efficient 3-party MPC for dynamic data structures. *Proc. Priv. Enhancing Technol.*, 2024(3):692–714, 2024.

[SVLH23] Kyle Storrier, Adithya Vadapalli, Allan Lyons, and Ryan Henry. Grotto: Screaming fast (2+1)-PC or F2n via (2, 2)-dpfs. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26-30, 2023*, pages 2143–2157. ACM, 2023.

[VBH21] Adithya Vadapalli, Fattaneh Bayatbabolghani, and Ryan Henry. You may also like... privacy: Recommendation systems meet PIR. *Proc. Priv. Enhancing Technol.*, 2021(4):30–53, 2021.

[VHG23] Adithya Vadapalli, Ryan Henry, and Ian Goldberg. Duoram: A bandwidth-efficient distributed ORAM for 2- and 3-party computation. In *32nd USENIX Security Symposium, USENIX Security 2023*, pages 3907–3924. USENIX Association, 2023.

[Wag22] Sameer Wagh. Pika: Secure computation using function secret sharing over rings. *Proc. Priv. Enhancing Technol.*, 2022(4):351–377, 2022.

[WGC19] Sameer Wagh, Divya Gupta, and Nishanth Chandran. Securenn: 3-party secure computation for neural network training. *Proc. Priv. Enhancing Technol.*, 2019(3):26–49, 2019.

[WNL+14] Xiao Shaun Wang, Kartik Nayak, Chang Liu, T.-H. Hubert Chan, Elaine Shi, Emil Stefanov, and Yan Huang. Oblivious data structures. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, Scottsdale, AZ, USA, November 3-7, 2014*, pages 215–226. ACM, 2014.

[Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th Annual Symposium on Foundations of Computer Science, Toronto, Canada, 27-29 October 1986*, pages 162–167. IEEE Computer Society, 1986.

[ZPZP25] Jinhao Zhu, Liana Patel, Matei Zaharia, and Raluca Ada Popa. Compass: Encrypted semantic search with high accuracy. In *19th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2025*, pages 915–938. USENIX Association, 2025.

[ZSF25] Mingxun Zhou, Elaine Shi, and Giulia Fanti. Pacmann: Efficient private approximate nearest neighbor search. In *The Thirteenth International Conference on Learning Representations, ICLR 2025*. OpenReview.net, 2025.